Отчет по домашнему заданию № 3 «MapReduce и DuckDB»

Анализ навыков и зарплат по вакансиям IT

Выполнил: Черников Кирилл Михайлович, группа J3310, вариант 17: «4. Программист, разработчик на C# и .NET»

Цель работы: поднять объектное хранилище RustFS, загрузить в него датасет вакансий, посчитать частоты навыков, пар и троек навыков своего варианта двумя способами (свой MapReduce и DuckDB), сверить результаты, найти навыки, которые повышают и понижают зарплату, сравнить три региона и замерить время и память.

Окружение: Windows 11, Docker Desktop, RustFS 1.0.1, Python 3.11, DuckDB 2.0.0, boto3, psutil. Код лежит в папке `src/`, полные выводы команд - в `outputs/`, результаты - в бакете `hh-lake` и в папке `results/`.

1. Объектное хранилище

Хранилище RustFS поднято по схеме из домашнего задания № 2: один узел, 4 папки-диска, Erasure Coding 2+2 (`infra/docker-compose.yml`). Стенд от ДЗ № 2 к этому моменту не сохранился (тома Docker были очищены), поэтому я поднял его заново тем же compose-файлом. Запуск:

```
docker compose -f infra/docker-compose.yml up -d     # RustFS: S3 API localhost:9000, консоль localhost:9001
```

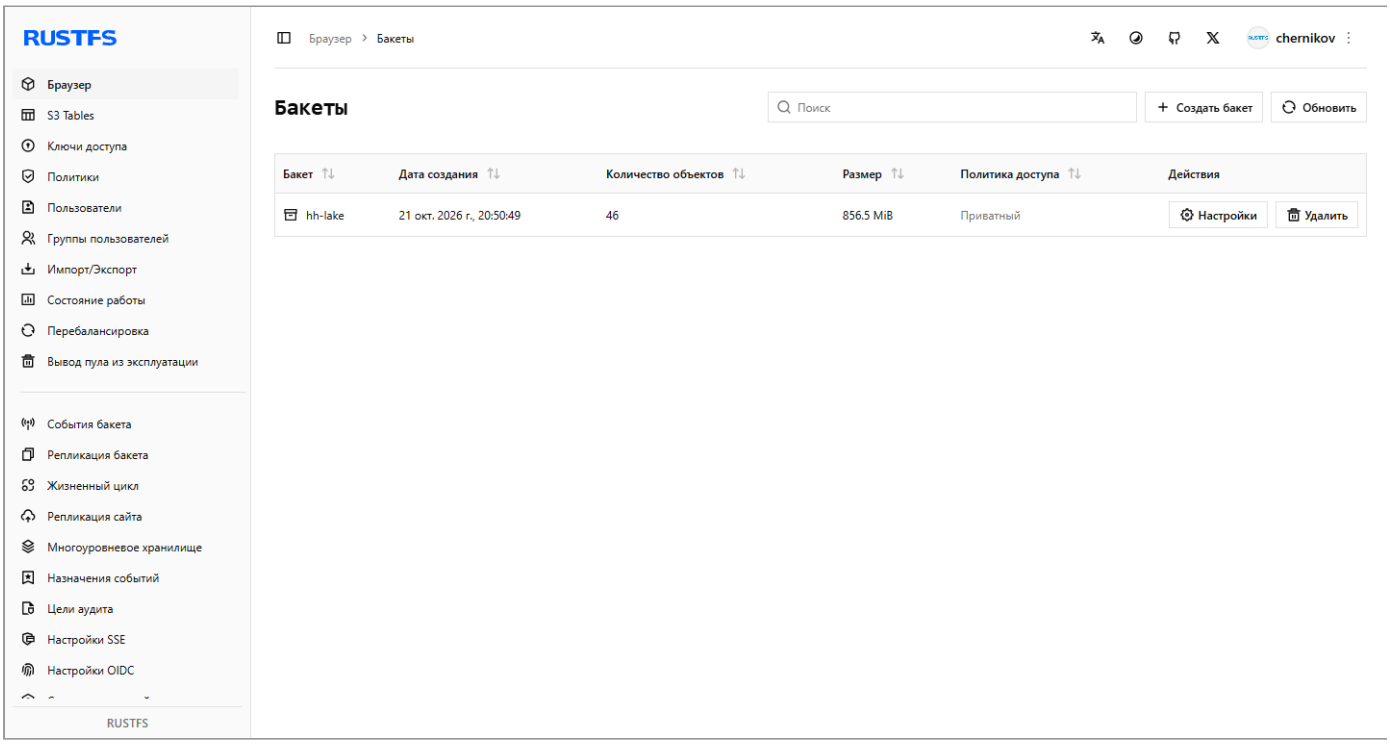


Рисунок 1 - Веб-консоль RustFS: бакет hh-lake

2. Загрузка данных

Датасет - снапшот вакансий hh.ru на середину 2024 года (`it_vacancies_2024.csv`, 847 619 384 байта, 229 388 вакансий, 23 колонки). Скрипт `src/upload.py` создает бакет `hh-lake` и загружает файл частями по 64 МиБ (multipart upload через boto3):

```
bucket hh-lake: создан
upload it_vacancies_2024.csv -> s3://hh-lake/raw/it_vacancies_2024.csv: 11.5 s
head_object: ContentLength=847619384, ETag="0e6d21890db664d9bf83e1427665b106-13", LastModified=2026-10-21 17:51:00+00:00
локальный размер: 847619384
  s3://hh-lake/raw/it_vacancies_2024.csv  847619384
```

Перед тем как резать файл на чанки, я проверил, нет ли в CSV полей с переводами строк внутри (например, в описании вакансии). Их нет: в файле 229 389 символов `\n` - ровно заголовок плюс 229 388 записей. Одна строка файла - одна вакансия, поэтому чанки можно резать по байтам, дочитывая последнюю строку, как это делает Hadoop.

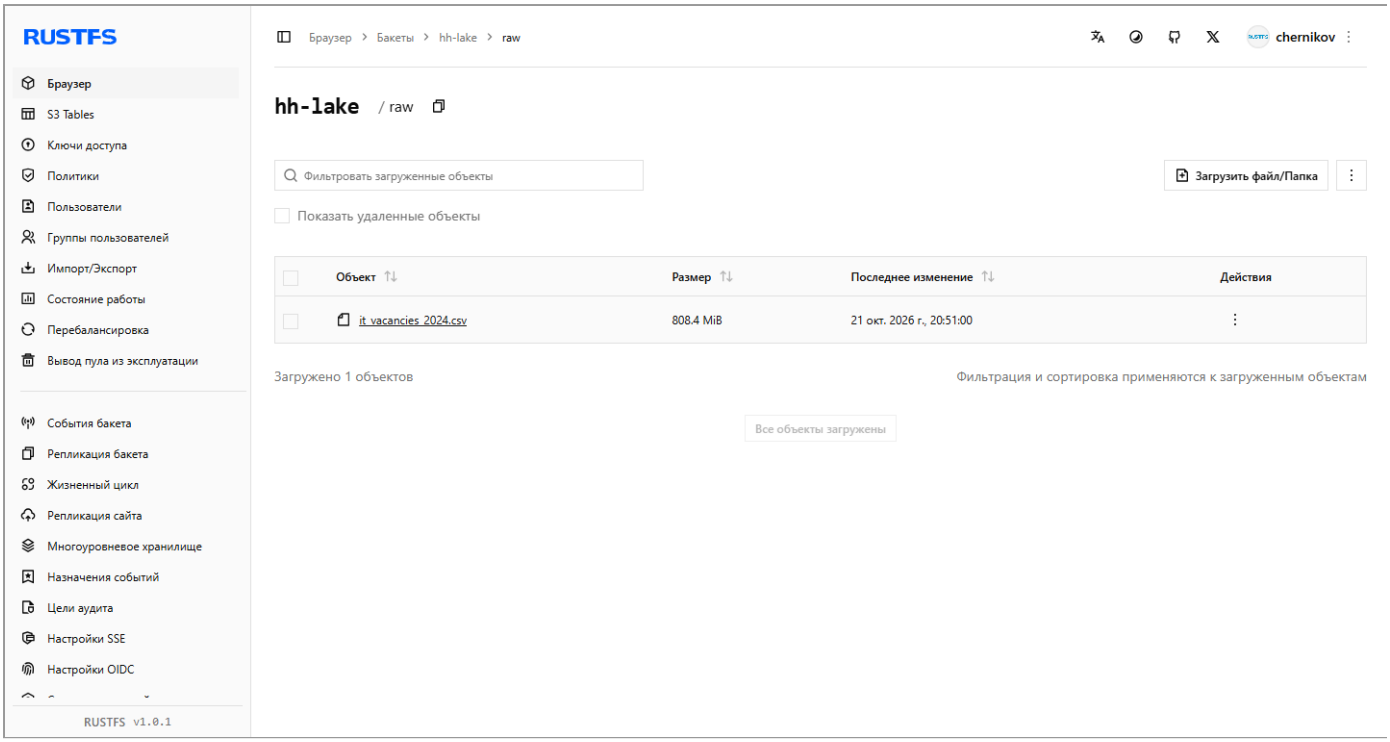


Рисунок 2 - Объект raw/it_vacancies_2024.csv в бакете hh-lake

3. Условие варианта

В таблице вариантов указан сегмент «Программист, разработчик на C# и .NET», точного правила отбора нет, поэтому условие я составил сам по данным (скрипты `explore/`). В датасете есть колонка `role`, и ее значение «Программист, разработчик» совпадает с формулировкой варианта. Внутри роли вакансии C#/.NET отличаются по названию:

```
role = 'Программист, разработчик'
AND regexp_matches(lower(name), '(c#|c#|\.net|dotnet)')     -- вторая «c#» написана кириллической буквой
```

Таблица 1 - Сравнение вариантов условия

Условие	Вакансий	С навыками	С зарплатой в RUR
роль + C#/.NET в названии (латинская и кириллическая «с») - итоговое	**4 356**	**3 345**	**1 446**
то же, но только латинская «с»	3 892	2 990	1 307
C#/.NET в названии, любая роль	4 686	3 596	1 485

Часть работодателей пишет «Программист C#» с русской буквой «С», и простое условие по `c#` теряет 464 вакансии (11%). Вакансии с другой ролью (тимлиды - 155, тестировщики - 140) я не беру, так как это другие профессии. Unity без упоминания C# тоже не беру - это отдельный сегмент (геймдев).

Навыки нормализуются одинаково в обоих способах: нижний регистр, обрезка пробелов, без повторов внутри вакансии. В процессе выяснилось, что та же проблема с кириллицей есть и в самих навыках: `c#` и `c#` (613 вхождений кириллицей), `.net core`, `c++`, `ci/cd`, `odata` и т.д. Поэтому добавлено правило: если в навыке из русских букв есть только буквы-двойники латиницы (а, с, е, о, р, х, к, у), они заменяются на латинские. Настоящие русские навыки («ооп», «работа в команде») содержат и другие буквы и не меняются. После этого `c#` вырос с 2 237 до 2 813 вхождений.

4. Свой MapReduce

Реализация - `src/mapreduce.py`, процессы Python (`ProcessPoolExecutor`):

1. Мастер узнает размер объекта (`head_object`) и делит его на сплиты по 64 МиБ - 13 map-задач.
2. Маппер читает свой сплит из S3 запросом `Range: bytes=start-end`. Граница - как в Hadoop: строка принадлежит сплиту, в котором она начинается. Маппер пропускает обрывок строки в начале сплита и дочитывает свою последнюю строку за его концом.
3. Маппер отбирает вакансии варианта, нормализует навыки и для каждой вакансии выдает все сочетания навыков размера 1, 2 и 3. Комбайнер суммирует их прямо в маппере (`Counter`).
4. При shuffle ключ попадает к редьюсеру номер `crc32(ключ) mod 4`, редьюсеры (4 процесса) суммируют частоты.

5. Результаты для трех размеров сочетаний записываются в `s3://hh-lake/results/mapreduce/`: `skills_k1.csv`, `pairs_k2.csv`, `triples_k3.csv` и `summary.json`.

```
object s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 bytes; splits: 13 x 64 MiB; mode=optimized; workers=4; reducers=
4
map: 7.53 s; shuffle+reduce: 0.48 s; total: 9.37 s
строк прочитано мапперами: 229389 (данных 229388); вакансий варианта: 4356; с навыками: 3345
пар (ключ, значение) от мапперов: 250377
k=1: уникальных ключей 1156, сумма частот 22829; топ-3: [['c#', 2813], ['.net framework', 1130], ['asp.net', 1093]]
k=2: уникальных ключей 16920, сумма частот 92551; топ-3: [['.net framework', 'c#', 1003], ['asp.net', 'c#', 914], ['c#',
'postgresql', 861]]
k=3: уникальных ключей 112376, сумма частот 309607; топ-3: [['asp.net', 'c#', 'postgresql', 352], ['.net framework', 'c
#', 'postgresql', 311], ['.net framework', 'asp.net', 'c#', 308]]
результаты: s3://hh-lake/results/mapreduce/
```

Мапперы вместе прочитали ровно 229 389 строк, то есть каждая строка файла обработана один раз и на стыках сплитов ничего не потерялось и не задвоилось. Для проверки стыков я также запустил расчет с неудобным размером сплита 7 МиБ (116 сплитов): результат совпал побайтно (`outputs/05b_chunk7.txt`).

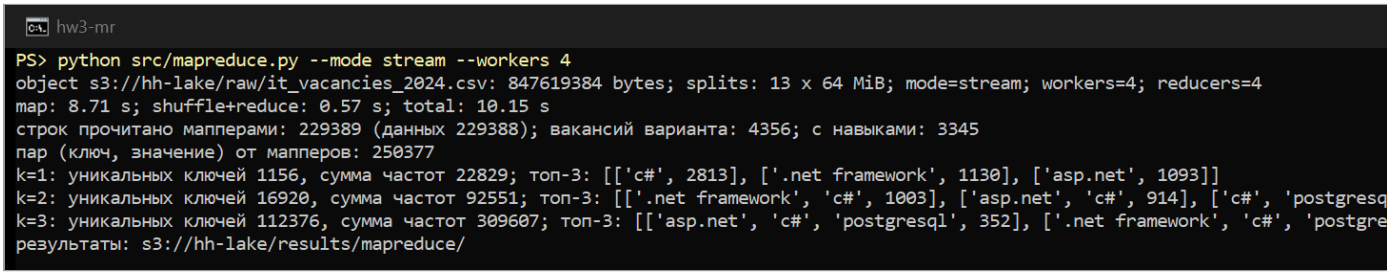


Рисунок 3 - Запуск MapReduce (режим stream, 4 маппера)

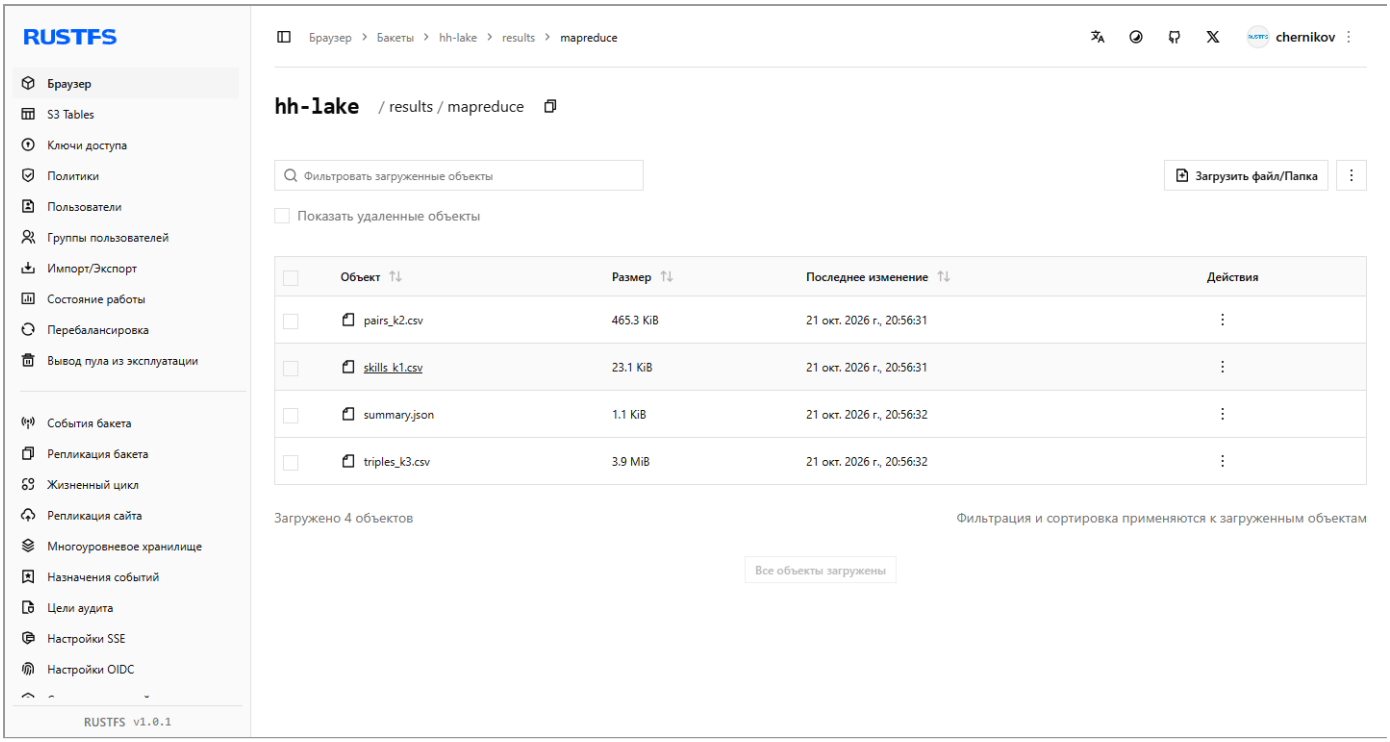


Рисунок 4 - Результаты MapReduce для трех размеров сочетаний в hh-lake

4.1. Отказ маппера

Флаг `--fail-task 3` заставляет маппер № 3 упасть на первой попытке. Мастер ловит ошибку и перезапускает задачу (до 3 попыток). Так как map-задача читает свой диапазон байтов заново и не имеет побочных эффектов, повторный запуск дает тот же результат - файлы совпали побайтно (SHA-256):

```
> python src/mapreduce.py --workers 4 --fail-task 3 --out-prefix mapreduce_failtest
object s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 bytes; splits: 13 x 64 MiB; mode=optimized; workers=4; reducers=
4
  map task 3: ОТКАЗ на попытке 1: имитация сбоя маппера 3 (попытка 1)
  map task 3: успешно с попытки 2
map: 7.86 s; shuffle+reduce: 0.54 s; total: 10.49 s

> сравнение результатов с обычным запуском (SHA-256 файлов)
skills_k1.csv     обычный 6494D84687724CCA  с отказом 6494D84687724CCA  совпадает: True
pairs_k2.csv      обычный 17F0BC26B0CE33D6  с отказом 17F0BC26B0CE33D6  совпадает: True
triples_k3.csv    обычный DF05762386EF5336  с отказом DF05762386EF5336  совпадает: True
```

```
hw3-fail
PS> python src/mapreduce.py --mode stream --workers 4 --fail-task 3 --out-prefix mapreduce_failtest
object s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 bytes; splits: 13 x 64 MiB; mode=stream; workers=4; reducers=4
  map task 3: ОТКАЗ на попытке 1: имитация сбоя маппера 3 (попытка 1)
  map task 3: успешно с попытки 2
map: 8.84 s; shuffle+reduce: 0.47 s; total: 10.47 s
строк прочитано мапперами: 229389 (данных 229388); вакансий варианта: 4356; с навыками: 3345
пар (ключ, значение) от мапперов: 250377
k=1: уникальных ключей 1156, сумма частот 22829; топ-3: [['c#', 2813], ['.net framework', 1130], ['asp.net', 1093]]
k=2: уникальных ключей 16920, сумма частот 92551; топ-3: [['.net framework', 'c#', 1003], ['asp.net', 'c#', 914], ['c#', 'postgresq
k=3: уникальных ключей 112376, сумма частот 309607; топ-3: [['asp.net', 'c#', 'postgresql', 352], ['.net framework', 'c#', 'postgre
результаты: s3://hh-lake/results/mapreduce_failtest/

PS> Get-FileHash results\mapreduce\*.csv vs results\mapreduce_failtest\*.csv
skills_k1.csv    обычный 6494D84687724CCA  с отказом 6494D84687724CCA  совпадает: True
pairs_k2.csv     обычный 17F0BC26B0CE33D6  с отказом 17F0BC26B0CE33D6  совпадает: True
triples_k3.csv   обычный DF05762386EF5336  с отказом DF05762386EF5336  совпадает: True
```

Рисунок 5 - Тест отказа маппера

5. DuckDB

Второй способ - `src/duckdb_skills.py`. DuckDB читает тот же объект прямо из RustFS через расширение `httpfs` (S3 secret с endpoint `localhost:9000`), условие варианта и нормализация навыков написаны на SQL. Пары и тройки получаются самосоединением таблицы навыков с условием `a.skill < b.skill < c.skill`, чтобы каждое сочетание считалось один раз:

```
CREATE TEMP TABLE skills AS
WITH v AS (SELECT vacancy_id, key_skills
           FROM read_csv('s3://hh-lake/raw/it_vacancies_2024.csv', header = true, all_varchar = true)
           WHERE role = 'Программист, разработчик' AND regexp_matches(lower(name), '(c#|c#|\.net|dotnet)'))
SELECT DISTINCT vacancy_id, <нормализация навыка> AS skill
FROM v, unnest(string_split(coalesce(key_skills, ''), ';')) AS t(s) WHERE lower(trim(s)) <> '';

SELECT a.skill, b.skill, c.skill, count(*) FROM skills a
JOIN skills b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
JOIN skills c ON b.vacancy_id = c.vacancy_id AND b.skill < c.skill
GROUP BY ALL;                        -- тройки; пары и навыки - аналогично
```

```
DuckDB 2.0.0; источник s3://hh-lake/raw/it_vacancies_2024.csv
чтение + фильтр варианта: 4.16 s; всего: 6.45 s
вакансий варианта: 4356; с навыками: 3345
k=1: уникальных ключей 1156, сумма частот 22829; топ-3: [('c#', 2813), ('.net framework', 1130), ('asp.net', 1093)]
k=2: уникальных ключей 16920, сумма частот 92551; топ-3: [('.net framework', 'c#', 1003), ('asp.net', 'c#', 914), ('c#',
'postgresql', 861)]
k=3: уникальных ключей 112376, сумма частот 309607; топ-3: [('asp.net', 'c#', 'postgresql', 352), ('.net framework', 'c
#', 'postgresql', 311), ('.net framework', 'asp.net', 'c#', 308)]
результаты: s3://hh-lake/results/duckdb/
```

```
hw3-duckdb
PS> python src/duckdb_skills.py
DuckDB 2.0.0; источник s3://hh-lake/raw/it_vacancies_2024.csv
чтение + фильтр варианта: 5.07 s; всего: 7.12 s
вакансий варианта: 4356; с навыками: 3345
k=1: уникальных ключей 1156, сумма частот 22829; топ-3: [('c#', 2813), ('.net framework', 1130), ('asp.net', 1093)]
k=2: уникальных ключей 16920, сумма частот 92551; топ-3: [('.net framework', 'c#', 1003), ('asp.net', 'c#', 914), ('c#', 'postgresq
k=3: уникальных ключей 112376, сумма частот 309607; топ-3: [('asp.net', 'c#', 'postgresql', 352), ('.net framework', 'c#', 'postgre
результаты: s3://hh-lake/results/duckdb/
```

Рисунок 6 - Запуск DuckDB

6. Сверка результатов

Скрипт `src/verify.py` скачивает оба набора результатов из `hh-lake` и сравнивает каждую строку: какие ключи есть только в одном наборе и у каких ключей разные числа. Расхождений нет ни на одном уровне:

```
k    MapReduce: ключей    сумма  DuckDB: ключей    сумма  только MR  только DDB  разные числа
1                 1156    22829             1156    22829          0           0             0
2                16920    92551            16920    92551          0           0             0
3               112376   309607           112376   309607          0           0             0
ИТОГ: результаты полностью совпадают
```

Таблица 2 - Контрольные числа

Показатель	MapReduce	DuckDB
вакансий варианта / с навыками	4 356 / 3 345	4 356 / 3 345
уникальных навыков / пар / троек	1 156 / 16 920 / 112 376	1 156 / 16 920 / 112 376
сумма частот навыков / пар / троек	22 829 / 92 551 / 309 607	22 829 / 92 551 / 309 607
топ-1 навык / пара / тройка	c# 2 813; .net framework + c# 1 003; asp.net + c# + postgresql 352	те же

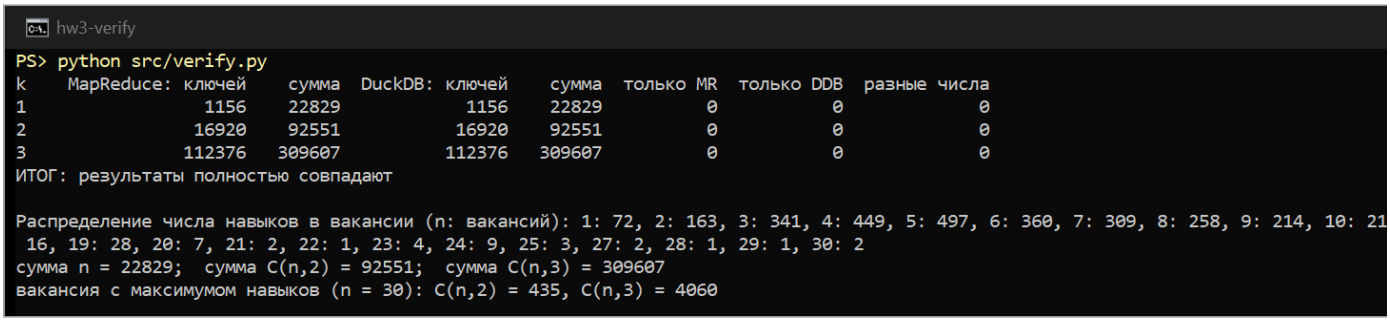


Рисунок 7 - Построчная сверка MapReduce и DuckDB

6.1. Почему пар и троек так много

Уникальных навыков 1 156, пар уже в 15 раз больше, а троек - почти в 100 раз. Причина в числе сочетаний внутри одной вакансии: вакансия с n навыками дает n навыков, C(n,2) = n*(n-1)/2 пар и C(n,3) = n*(n-1)*(n-2)/6 троек. Это проверено точно: сумма C(n,2) по всем вакансиям совпадает с суммой частот пар, а сумма C(n,3) - с суммой частот троек:

```
Распределение числа навыков в вакансии (n: вакансий): 1: 72, 2: 163, 3: 341, 4: 449, 5: 497, 6: 360, 7: 309, 8: 258, 9: 2
14, 10: 214, 11: 111, 12: 96, 13: 44, 14: 46, 15: 34, 16: 40, 17: 21, 18: 16, 19: 28, 20: 7, 21: 2, 22: 1, 23: 4, 24: 9,
25: 3, 27: 2, 28: 1, 29: 1, 30: 2
сумма n = 22829;  сумма C(n,2) = 92551;  сумма C(n,3) = 309607
вакансия с максимумом навыков (n = 30): C(n,2) = 435, C(n,3) = 4060
```

Медиана - 6 навыков на вакансию (15 пар, 20 троек), но рост дают «длинные» вакансии: две вакансии с 30 навыками дают по 4 060 троек каждая, это больше, чем все 341 вакансия с 3 навыками вместе. Число уникальных сочетаний тоже растет, потому что редкие навыки образуют много редких сочетаний: 112 376 разных троек при 309 607 вхождениях, то есть большинство троек встречается 1-3 раза.

7. Профилирование и оптимизация (бонус)

Скрипт `src/profile_runs.py` запускает каждую конфигурацию отдельным процессом 3 раза. Время меряется по часам, пик памяти - как максимум суммарного RSS всего дерева процессов (мастер плюс мапперы), снимается каждые 25 мс через psutil. Сравнивались три версии MapReduce:

- `naive` («до»): маппер разбирает каждую строку файла как CSV и выдает пары (ключ, 1) без комбайнера, shuffle - сортировка всех пар, один редьюсер;
- `optimized`: дешевый предфильтр строки до разбора CSV, комбайнер в маппере, hash-партиционирование;
- `stream` («после»): сплит читается из S3 потоком по 1 МиБ, строки отбираются по сырым байтам до декодирования, в строку Python превращаются только около 2% строк.

```
конфигурация  команда                                          время, с (мин/медиана/макс)   пик памяти, МиБ (медиа
на)
mr_naive_w1   src/mapreduce.py --mode naive --workers 1          24.38 / 24.43 / 25.10                  472.9
mr_opt_w1     src/mapreduce.py --workers 1                       19.54 / 23.86 / 25.97                  436.2
mr_opt_w4     src/mapreduce.py --workers 4                       13.46 / 13.75 / 14.59                 1262.9
mr_stream_w1  src/mapreduce.py --mode stream --workers 1         23.42 / 26.74 / 28.15                  195.6
mr_stream_w4  src/mapreduce.py --mode stream --workers 4         11.68 / 12.04 / 17.61                  384.8
duckdb        src/duckdb_skills.py                                6.81 /  6.81 /  9.20                 1269.8
```

```
mr_stream_w8  src/mapreduce.py --mode stream --workers 8         10.80 / 11.07 / 12.08                  616.3
```

Таблица 3 - Время и память (3 запуска, медиана)

Конфигурация	**Время, с (мин / медиана / макс)**	**Пик памяти, МиБ**
MapReduce naive, 1 процесс («до»)	24,38 / 24,43 / 25,10	473
MapReduce optimized, 1 процесс	19,54 / 23,86 / 25,97	436
MapReduce optimized, 4 процесса	13,46 / 13,75 / 14,59	1 263
MapReduce stream, 1 процесс	23,42 / 26,74 / 28,15	196
MapReduce stream, 4 процесса («после»)	**11,68 / 12,04 / 17,61**	**385**
MapReduce stream, 8 процессов	10,80 / 11,07 / 12,08	616
DuckDB	6,81 / 6,81 / 9,20	1 270

Больше всего оптимизация повлияла на память. Версии naive и optimized загружают весь сплит 64 МиБ в память, переводят его в строку Python и режут на список строк - около 430 МиБ на процесс. Нужны из них только 2% строк. Потоковое чтение снизило пик в 1 процессе с 473 до 196 МиБ (-59%), а при 4 процессах - с 1 263 до 385 МиБ (-70%) по сравнению с optimized.

По времени в одном процессе все версии работают примерно одинаково, 24-27 с. Это скорость чтения 848 МБ из RustFS одним потоком - около 35 МБ/с, и задача упирается в чтение данных. Время сокращает только параллельное чтение: 4 маппера - 12,0 с (в 2 раза быстрее, чем «до»). При 8 мапперах выигрыш почти пропал (11,1 с), а память выросла - упираемся в пропускную способность одного узла RustFS, у которого все 4 диска лежат на одном физическом диске.

До оптимизации было 24,4 с и 473 МиБ, после - 12,0 с и 385 МиБ, а файлы результатов совпадают побайтно. DuckDB быстрее в 1,8 раза (6,8 с), но использует 1,27 ГиБ: он читает файл несколькими параллельными запросами и разбирает CSV векторно на C++, держа данные в своем буфере. MapReduce на Python экономнее по памяти. DuckDB быстрее, и кода для него нужно меньше: один SQL-запрос вместо своего планировщика.

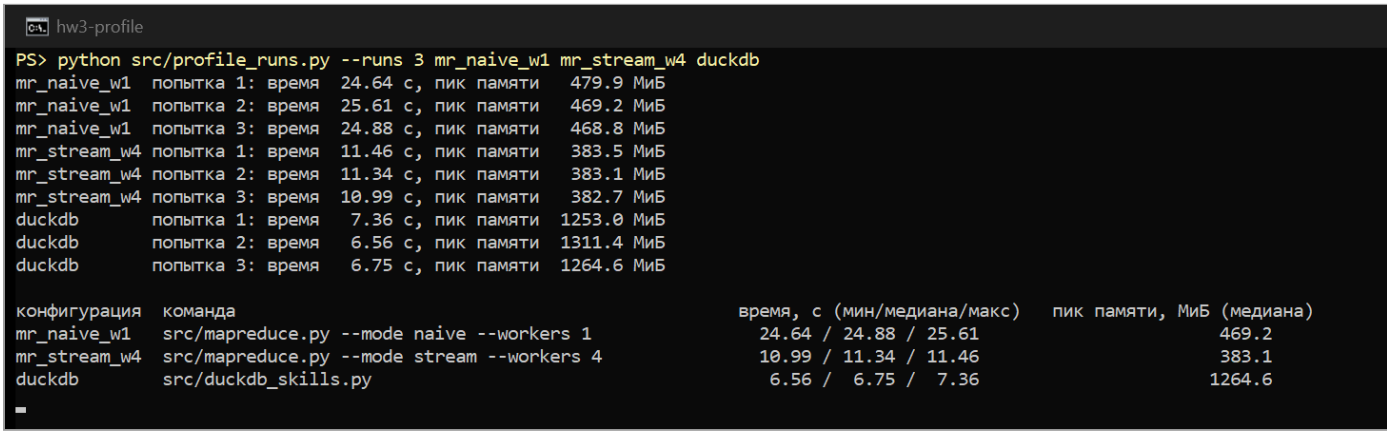


Рисунок 8 - Профилирование: 3 запуска каждой конфигурации

8. Навыки и зарплата

Код - `src/salary_analysis.py`. Зарплата вакансии: только рубли, середина вилки «от-до» или одна указанная граница; если зарплата указана до вычета налога (gross), она умножается на 0,87, чтобы все было «на руки»; значения вне 15 000 - 1 000 000 ₽ отброшены. Прибавка пары считается от медианы **более частого** навыка пары (то есть это прибавка от добавления менее частого навыка), прибавка тройки - от медианы самой частой из трех входящих в нее пар. В рейтинг попадают пары, встречающиеся не меньше 25 раз, и тройки - не меньше 20 раз.

8.1. Ловушка: массовые публикации

Первая версия расчета выдала странный результат: у всех лучших пар медиана ровно 300 000 ₽, и почти все они связаны с ООП (SOLID + ООП, DDD + ООП, Clean Architecture + ООП). Проверка показала, что 35 из 62 вакансий с парой SOLID + ООП разместил один работодатель (employer_id 1136961) с одинаковой вилкой 280-350 тыс. и всего двумя разными названиями - это одна вакансия, опубликованная во многих городах. У DDD + ООП 38 вакансий приходятся всего на 3 работодателей:

```
ПАРЫ, которые сильнее всего ПОВЫШАЮТ медиану (всего сочетаний, прошедших пороги: 268):
  сочетание                                n раб.   медиана  база (более частая)              n   медиана  + нав
ык               прибавка      %
  solid + ооп                             62   11   300 000  ооп                            180   157 500  solid
142 500  90.5%
  .net + ооп                              59    8   300 000  ооп                            180   157 500  .net
142 500  90.5%
```

```
  ddd + ооп                                38    3   300 000  ооп                                180   157 500  ddd
142 500  90.5%
  clean architecture + ооп                 36    2   300 000  ооп                                180   157 500  clean
architecture     142 500  90.5%
```

Поэтому в итоговом расчете одинаковые вакансии одного работодателя (то же название, зарплата и навыки) считаются один раз (было 1 442 вакансии, стало 1 071), а сочетание должно встречаться минимум у 10 разных работодателей для пар и у 8 - для троек. Общая медиана по варианту - 160 000 ₽.

8.2. Итоговые таблицы

Таблица 4 - Пары, которые сильнее всего повышают медиану

Пара	n	Работ.	Медиана	База (более частый)	n базы	Медиана базы	Прибавка
.net core + mongodb	34	13	300 000	.net core	212	200 000	+100 000 (+50%)
c# + kubernetes	35	26	250 000	c#	781	160 000	+90 000 (+56%)
c# + ci/cd	25	15	250 000	c#	781	160 000	+90 000 (+56%)
ms sql + rabbitmq	28	15	255 500	ms sql	194	174 000	+81 500 (+47%)
mongodb + ms sql	27	14	250 000	ms sql	194	174 000	+76 000 (+44%)
entity framework + rabbitmq	38	29	252 500	entity framework	160	180 000	+72 500 (+40%)
kubernetes + postgresql	27	21	275 000	postgresql	278	202 500	+72 500 (+36%)
docker + entity framework	37	27	250 000	entity framework	160	180 000	+70 000 (+39%)
c# + mongodb	65	29	225 000	c#	781	160 000	+65 000 (+41%)
c# + kafka	37	25	225 000	c#	781	160 000	+65 000 (+41%)

Таблица 5 - Пары, которые сильнее всего понижают медиану

Пара	n	Работ.	Медиана	База (более частый)	n базы	Медиана базы	Прибавка
css + html	42	32	83 500	html	73	150 000	-66 500 (-44%)
c# + ms visual studio	55	41	100 000	c#	781	160 000	-60 000 (-38%)
c# + unity	45	37	100 000	c#	781	160 000	-60 000 (-38%)
javascript + postgresql	31	25	150 000	postgresql	278	202 500	-52 500 (-26%)
ms sql + ооп	33	22	121 800	ms sql	194	174 000	-52 200 (-30%)
css + javascript	35	25	100 000	javascript	132	150 000	-50 000 (-33%)
.net core + javascript	31	29	150 000	.net core	212	200 000	-50 000 (-25%)
c# + c++	34	23	115 000	c#	781	160 000	-45 000 (-28%)
.net framework + ms visual studio	28	22	120 900	.net framework	301	165 000	-44 100 (-27%)

Таблица 6 - Тройки: база - самая частая пара внутри тройки, прибавка - от третьего навыка

Тройка	n	Работ.	Медиана	База (самая частая пара)	n базы	Медиана базы	+ навык	Прибавка
повышают								
.net core + asp.net + ms sql	26	13	280 500	asp.net + ms sql	82	155 000	.net core	+125 500
.net core + c# + mongodb	30	10	300 000	.net core + c#	185	191 400	mongodb	+108 600
c# + mongodb + ms sql	23	11	275 000	c# + ms sql	177	174 000	mongodb	+101 000
asp.net + entity framework + rabbitmq	24	18	250 000	asp.net + entity framework	86	160 000	rabbitmq	+90 000
c# + ms sql + rabbitmq	25	13	261 000	c# + ms sql	177	174 000	rabbitmq	+87 000
c# + kubernetes + postgresql	24	18	268 000	c# + postgresql	244	200 000	kubernetes	+68 000
понижают								
css + html + javascript	29	21	80 000	html + javascript	49	150 000	css	-70 000
.net core + .net framework + postgresql	21	18	180 000	.net core + postgresql	100	250 000	.net framework	-70 000
c# + css + html	38	28	83 500	c# + html	68	150 000	css	-66 500
c# + git + html	21	17	87 000	c# + git	248	153 300	html	-66 300
c# + javascript + postgresql	24	21	144 600	c# + postgresql	244	200 000	javascript	-55 400
c# + ms sql + ооп	30	19	125 900	c# + ms sql	177	174 000	ооп	-48 100

Повышают зарплату навыки инфраструктуры нагруженного бэкенда: MongoDB, Kubernetes, CI/CD, RabbitMQ, Kafka, Docker. Медиана у вакансий с C# 160 тыс. Если к C# добавляется Kubernetes или CI/CD, медиана 250 тыс. Такие требования пишут в вакансиях для микросервисов и высоких нагрузок, где нужен опыт уровня middle+ и senior. Тот же смысл у троек: .NET Core к связке ASP.NET + MS SQL добавляет 125 тыс. - это переход со старого .NET Framework на современный стек.

Понижают зарплату навыки, которые указывают на более простые или специфичные позиции:

- CSS, HTML и JavaScript (до -44%) - это вакансии fullstack в небольших компаниях и стажерские позиции, где C#-разработчик еще и верстает;
- MS Visual Studio (-38%) - указание IDE в навыках типично для вакансий начального уровня и десктопной разработки; опытному разработчику IDE в требованиях не пишут;
- Unity (-38%) - геймдев традиционно платит меньше бизнес-разработки;
- C++ (-28%) - промышленные и встраиваемые системы, где C# - вспомогательный язык;
- ООП как отдельный навык (-28% и -30%) - базовое знание, которое отдельно требуют в вакансиях для начинающих;
- .NET Framework рядом с .NET Core и PostgreSQL (-28%) - поддержка старого кода (легаси).

Скорее всего, такие навыки просто показывают, какого уровня позиция. По этим данным нельзя сказать, что навык сам меняет зарплату.

9. Региональный анализ

Для сравнения я взял Москву и Санкт-Петербург - два главных рынка IT - и Самару, мой родной город, чтобы понять, как выглядит рынок C#/.NET в регионе по сравнению со столицами. В Самаре всего 69 вакансий и только 16 с зарплатой, поэтому выводы по ней ориентировочные. Код - `src/regions.py`, зарплаты считаются так же, как в разделе 8 (без дублей).

Таблица 7 - Рынок и зарплаты

Показатель	Москва	Санкт-Петербург	Самара	Вся Россия
вакансий (доля от всех вакансий варианта)	1 848 (42,4%)	554 (12,7%)	69 (1,6%)	4 356
работодателей	710	251	42	1 416
опыт: нет опыта / 1-3 года / 3-6 лет / более 6	2,8 / 23,4 / 62,8 / 11,0%	3,4 / 28,5 / 57,4 / 10,6%	8,7 / 44,9 / 39,1 / 7,2%	4,9 / 33,0 / 53,7 / 8,4%
удаленная работа	45,7%	33,9%	29,0%	37,8%
вакансий с зарплатой (без дублей)	434	146	16	1 071
медиана зарплаты, ₽	**217 500**	**180 000**	**98 750**	**160 000**
25-й / 75-й перцентиль, ₽	160 000 / 275 000	120 000 / 250 000	70 000 / 125 062	100 000 / 241 800
медиана, опыт 1-3 года / 3-6 лет, ₽	150 000 / 240 000	120 000 / 205 000	98 750 / -	120 000 / 200 000
медиана: удаленка (вакансий) / офис, ₽	225 000 (237) / 200 000	225 000 (60) / 165 150	304 500 (1) / 97 500	209 400 (402) / 143 550

Таблица 8 - Востребованность навыков (доля вакансий региона с указанными навыками)

	Москва (1 378 вакансий с навыками)	Санкт-Петербург (421)	Самара (58)
навыки	c# 82%, asp.net 34%, postgresql 33%, .net framework 32%, sql 25%, git 25%, .net core 24%, ms sql 20%, docker 15%	c# 84%, .net framework 34%, postgresql 29%, asp.net 29%, sql 29%, .net core 23%, git 20%, ms sql 19%, rabbitmq 18%	c# 79%, .net framework 38%, .net core 31%, ms sql 31%, asp.net 29%, postgresql 28%, git 26%, sql 22%, javascript 17%
пары	.net framework + c# 28%, c# + postgresql 28%, asp.net + c# 26%, c# + git 22%, c# + sql 22%	.net framework + c# 30%, c# + sql 27%, asp.net + c# 24%, c# + postgresql 23%, .net core + c# 19%	.net framework + c# 34%, .net core + c# 31%, c# + ms sql 29%, c# + git 24%, c# + postgresql 22%
тройки	asp.net + c# + postgresql 11%, c# + git + postgresql 9%, .net framework + c# + git 9%	.net framework + c# + sql 11%, .net framework + c# + postgresql 10%, c# + postgresql + sql 10%	.net framework + c# + ms sql 19%, .net core + c# + git 17%, .net core + c# + postgresql 14%
отличительные навыки (доля в регионе / доля по России)	signalr x2,3; k8s x2,3; prometheus x2,2; cqrs x1,9	avalonia x3,8; autodesk revit x3,4; .net 6 x3,0; design patterns x2,3	orm x4,9; субд x3,0; angular x2,4; ms sql x1,5

Таблица 9 - Медиана зарплаты по ключевым навыкам, ₽ (в скобках - число вакансий)

Навык	Москва	Санкт-Петербург	Самара	Вся Россия
c#	208 800 (314)	180 000 (114)	100 000 (11)	160 000 (781)
.net core	250 000 (105)	200 000 (27)	158 000 (4)	200 000 (212)
.net framework	217 500 (111)	180 000 (43)	107 500 (6)	165 000 (301)
postgresql	245 000 (135)	202 500 (34)	170 500 (4)	202 500 (278)
ms sql	220 000 (83)	220 000 (27)	117 450 (7)	174 000 (194)
docker	250 000 (62)	170 000 (15)	- (0)	205 000 (105)
kubernetes	280 000 (29)	250 000 (3)	- (0)	255 500 (40)
javascript	180 000 (51)	150 000 (20)	55 000 (3)	150 000 (132)

Москва - почти половина рынка (42% всех вакансий C#/.NET) и самые высокие зарплаты: медиана 217,5 тыс., на 36% выше общей. Рынок ориентирован на опытных разработчиков (63% вакансий - опыт 3-6 лет), а

отличительные навыки - SignalR, Kubernetes, Prometheus, CQRS - это облачный и нагруженный бэкенд. Docker и Kubernetes в Москве встречаются чаще и оплачиваются лучше (250-280 тыс.).

Петербург дешевле Москвы на 17% (медиана 180 тыс.), здесь чаще встречается .NET Framework, а отличительные навыки - Avalonia (кроссплатформенный десктоп), Autodesk Revit (плагины для программ инженерного проектирования), .NET 6. Это продуктовые компании, которые делают настольные и инженерные программы.

В Самаре медиана 98,8 тыс., на 55% ниже Москвы, почти половина вакансий для опыта 1-3 года, а вакансий без опыта втрое больше, чем в Москве (8,7% против 2,8%). Стек - корпоративные веб-системы: .NET + MS SQL + ORM + Angular. По Docker и Kubernetes в Самаре нет ни одной вакансии с зарплатой. Это в основном рынок для начинающих.

У удаленных вакансий в Москве и Петербурге одинаковая медиана 225 тыс., в офисе разница 35 тыс. Работодателю неважно, где живет удаленный разработчик, поэтому для разработчика из Самары удаленная работа на московскую компанию - способ получать в 2 с лишним раза больше местной медианы. (Медиана удаленки в самой Самаре - 304,5 тыс., но это одна вакансия, по ней вывод делать нельзя.)

Вакансий на одного работодателя по России 3,08, а в Самаре 1,64: в регионах меньше крупных компаний, которые публикуют вакансию сразу во многих городах (как работодатель из раздела 8.1).

Выводы

MapReduce и DuckDB дали одинаковый результат на всех трех уровнях. Пар и троек получилось намного больше, чем навыков, потому что вакансия с n навыками дает n*(n-1)/2 пар и еще больше троек, а несколько вакансий с 25-30 навыками дают тысячи сочетаний.

Была проблема с буквой «С». Часть работодателей пишет «C#» русской буквой «С», на экране она выглядит так же, как латинская «C». Из-за этого условие по «c#» сначала теряло 464 вакансии (11% варианта), а в навыках «c#» и «c#» считались двумя разными навыками: кириллическая версия встречалась 613 раз. Такие же «двойники» нашлись у «.net core», «c++», «ci/cd» и других. Я заметил это только тогда, когда в списке навыков оказалось два одинаковых на вид «c#». После исправления «c#» вырос с 2 237 до 2 813 вхождений. MapReduce и DuckDB совпадали и до исправления, потому что оба считали одинаково неправильно. Так что совпадение двух способов не защищает от ошибки в самих данных.

По профилированию видно, что мой MapReduce упирается в скорость чтения из хранилища, поэтому время сокращается только за счет параллельных мапперов. Потоковое чтение в основном помогло с памятью: при 4 мапперах пик снизился с 1 263 до 385 МиБ. В сумме с распараллеливанием время сократилось с 24,4 до 12,0 с, а память с 473 до 385 МиБ. DuckDB быстрее (6,8 с), потому что читает файл параллельно и разбирает CSV на C++. Памяти он использует больше, 1,27 ГиБ.

Больше всего платят в вакансиях, где нужен опыт с инфраструктурой (MongoDB, Kubernetes, очереди сообщений). Меньше платят там, где C#-разработчик еще и верстает или работает с Unity. Москва платит больше всех, у удаленных вакансий медиана в Москве и Петербурге одинаковая.